Documentación general del sistema Lisart

Qué se construyó, para qué sirve, cómo está armado por dentro, qué tan difícil fue cada pieza y qué tan fácil es explicarla. Es el documento de referencia para entender el sistema completo sin abrir el código.

Actualizada el 28 de septiembre de 2026

Este documento se genera desde `docs/documentacion_general.md`. No se edita a mano: se regenera con `lisart guia general`.

Este documento es el complemento técnico-operativo de la *Documentación General de Desarrollos y Proyectos Operativos de Lisart*. Aquel describe el panorama de todos los frentes de la escuela — formularios, mensajería, pagos, CRM—; éste describe **uno solo de ellos, a fondo**: el sistema administrativo que hoy sostiene los datos de Lisart. Donde aquel decía *"falta una estructura central de datos o repositorio operativo claro"*, esto es esa estructura, ya construida.

Se lee de dos maneras. Las secciones 1 a 4 cuentan qué es y qué resuelve, sin tecnicismos: son para dirección. De la 5 en adelante se entra en el detalle de cada pieza, con su dificultad y su explicación en lenguaje llano. La sección 15 es el glosario, y traduce cualquier palabra que suene rara.

Contenido

1. Resumen ejecutivo

2. El problema que resuelve

3. Cómo está armado, en una página

4. Cómo leer el detalle: la escala de dificultad

5. El modelo de datos: 17 tablas

6. La capa que cuida los archivos

7. De dónde salen los datos

8. Los procesos: de datos crudos a respuestas

9. Las pantallas

10. La documentación, que se genera sola

11. Cómo se protege el dato

12. Dónde estuvo lo difícil de verdad

13. Qué está verificado y qué no

14. Estado actual y siguientes pasos

15. Palabras que se usan aquí

16. Conclusión

1. Resumen ejecutivo

Lisart ya tenía captura digital: los formularios de Jotform recogen inscripciones, autorizaciones, firmas y comprobantes. Lo que no tenía era **un lugar donde esa información viviera ordenada, verificada y lista para usarse**. Los datos existían, pero cada consulta —cuántos alumnos hay, quién es tutor de quién, cuánto se debería cobrar este mes— seguía siendo trabajo manual.

Eso es lo que se construyó. El sistema toma los datos de las fuentes que ya existen, los limpia, los organiza en tablas con reglas claras, los valida y los convierte en respuestas concretas.

Lo que hay hoy, funcionando y con datos reales:

17 tablas con tipos, llaves y validación	La base de datos ordenada de la escuela
240 personas, 115 alumnos, 194 relaciones	Cargadas desde las inscripciones reales de Jotform
Una proyección de pago mensual	$148,655.00 sobre 98 de 115 alumnos, calculada sola
Dos pantallas para ver y editar	Sin tocar archivos ni saber programar
13 comandos	Cada tarea recurrente en una línea
208 pruebas automáticas	Que avisan si un cambio rompe algo
Respaldo antes de cada escritura	Nada se sobrescribe sin dejar copia

Lo que esto cambia en la práctica. Antes: un Excel que alguien mantenía a mano, con el riesgo de que una fila mal copiada se propagara sin que nadie lo notara. Ahora: los datos entran por un solo camino, el sistema se niega a escribir algo que no cuadre, y cualquier cifra que salga se puede rastrear hasta el envío de formulario que la originó.

Lo que falta. El sistema ya sostiene la información de **alumnos**. No sostiene todavía la de **pagos reales** — la proyección dice cuánto *debería* entrar, no cuánto entró— ni la de **profesores y clases**, que hoy sólo existen como datos de prueba. Las dos cosas dependen de una fuente que aún no está conectada (Kydemy) y de datos que sólo la escuela puede dar. Ese es el siguiente tramo, y está detallado en la sección 14.

Una decisión que conviene entender desde el principio: la base de datos son archivos CSV, es decir, hojas de cálculo. No es una carencia, es una elección. A escala de cientos de personas, un CSV disciplinado se abre en Excel, se respalda copiando una carpeta, no cobra renta mensual y no deja a la escuela atada a un proveedor. Todo el trabajo de ingeniería consistió, precisamente, en ponerle a esos archivos la disciplina que de fábrica no traen.

2. El problema que resuelve

El diagnóstico del documento anterior era la **dispersión**: cuando los datos de alumnos, pagos, autorizaciones y solicitudes viven en medios distintos, cada pregunta cuesta trabajo humano. Pero hay un segundo problema, menos visible y más caro, que aparece en cuanto los datos sí están juntos: **la información junta pero sin reglas se corrompe sola.**

Cuatro ejemplos concretos, todos observados en los datos reales de Lisart:

Una persona duplicada. En las 115 inscripciones hay **312 menciones de personas**: el alumno, su tutor, su contacto de emergencia, quien está autorizado a recogerlo. Muchas de esas menciones son *la misma persona en papeles distintos*: el 25 % de los tutores es además contacto de emergencia, y 7 alumnos son además tutores de alguien más. Sin un criterio de identidad, esas 312 menciones se habrían vuelto **312 personas**, cuando en realidad son 240. Los 72 humanos de más habrían recibido mensajes duplicados y habrían aparecido como deudores fantasma en cualquier corte de caja.

Un dato correcto en la casilla equivocada. Al revisar las fechas de nacimiento se encontró que varios tutores tenían menos de 18 años. No era un error de captura al azar: en el bloque de datos del tutor, algunas personas escribían la fecha **del hijo**. En 5 envíos la fecha del tutor es idéntica a la del alumno. Un sistema ingenuo hubiera guardado eso sin decir nada.

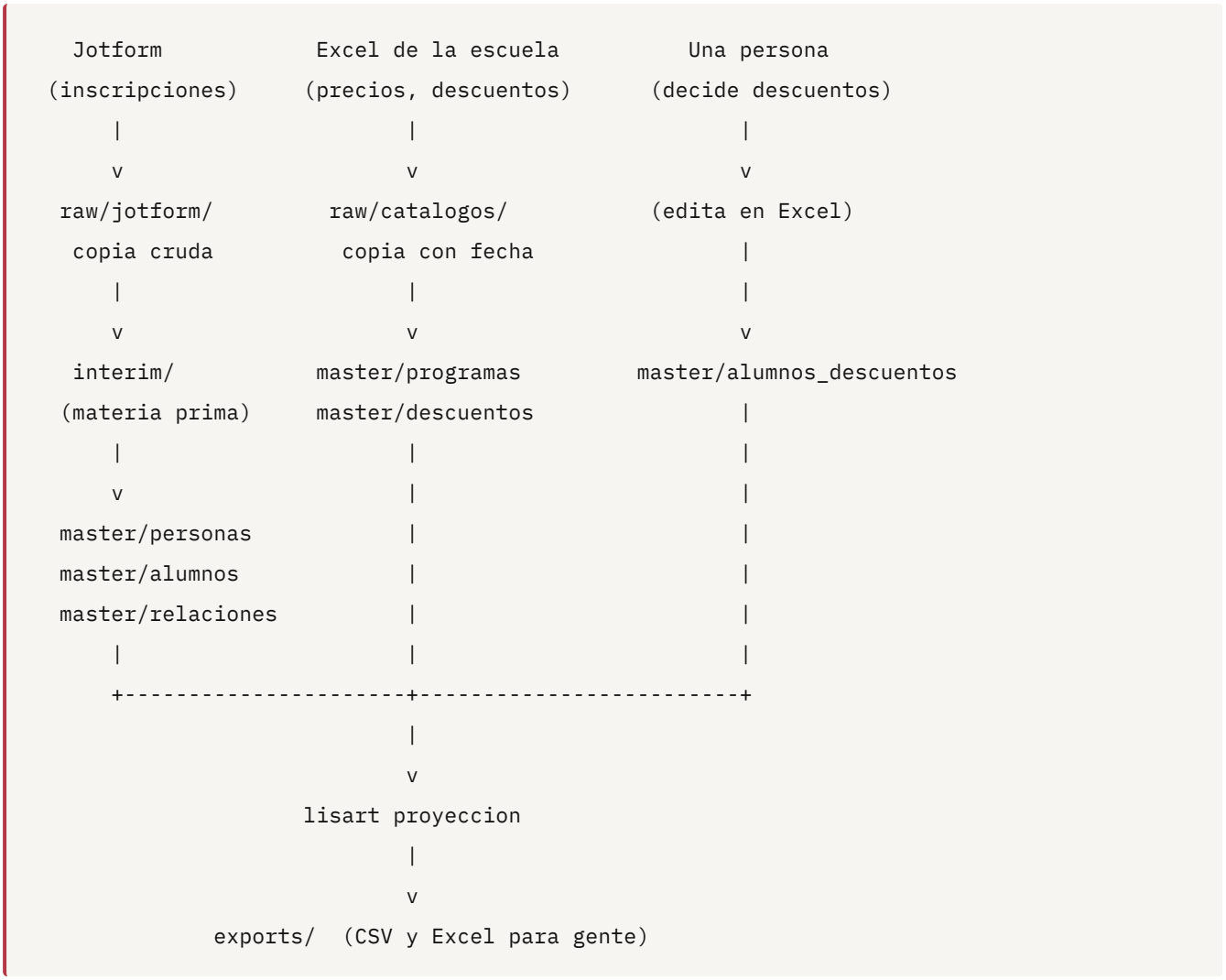
Un catálogo que se llena solo de variantes. Si el sistema diera de alta un instrumento cada vez que ve uno nuevo en un formulario, en seis meses el catálogo tendría "Guitarra", "guitarra", "Guitarra eléctrica" y "Gitarra" como cuatro instrumentos distintos, y ningún reporte volvería a cuadrar.

Un archivo escrito a medias. Si se corta la luz mientras se guarda un CSV de 243 personas, el resultado no es "se perdió el cambio": es un archivo con 130 filas y media, que Excel abre sin protestar.

El sistema está diseñado alrededor de estos cuatro riesgos. Su principio de fondo es **preferir no hacer nada antes que hacer algo dudoso**: si un dato no cuadra, se detiene y lo reporta, en vez de escribir a medias o inventar.

3. Cómo está armado, en una página

Los datos siguen siempre el mismo camino, y **nunca van en sentido contrario**:



Las cuatro capas y por qué existen:

CAPA	QUÉ GUARDA	REGLA
`raw`	La copia tal como llegó de la fuente	**Nunca se modifica.** Es la evidencia
`interim`	Materia prima ya aplanada, sin limpiar	Se puede rehacer entera desde `raw`
`master`	La verdad de la escuela	Lo único que se respalda
`exports`	Reportes para gente	**Desechable.** Se regenera cuando sea

La razón de que `raw` sea intocable es simple: es lo único que permite reconstruir todo si se descubre que un paso intermedio estaba mal. De hecho ya se usó de verdad —cuando se corrigió el mapeo de campos del formulario, la tabla se rehízo desde los archivos guardados sin volver a llamar a Jotform.

Tres ideas que explican el resto del sistema:

1. `master/` **es la verdad.** Es lo único que se respalda y lo único en lo que se confía para un reporte.
2. **Nada llega a** `master/` **sin pasar por la capa de código**, que le pone tipos, llaves y validación a unos archivos que de fábrica no traen nada de eso.
3. **Lo derivable se genera; lo que es conocimiento humano se respeta.** La documentación técnica, el mapa de tablas y los reportes se regeneran con un comando. Lo que decide una persona —qué descuento tiene quién, cómo se llama un salón— se escribe a mano y el código no lo pisa.

4. Cómo leer el detalle: la escala de dificultad

De aquí en adelante cada pieza lleva dos marcas. No son adorno: dicen dónde está el trabajo real y dónde está el riesgo de que algo se pierda cuando cambie quien mantiene el sistema.

Dificultad de construcción — cuánto costó, no en horas sino en decisiones que no eran obvias:

MARCA	SIGNIFICA
Baja	Trabajo directo. Se sabía qué había que escribir antes de empezar
Media	Hubo que decidir entre alternativas con consecuencias distintas
Alta	Se intentó de una forma, falló, y la solución salió de entender por qué

Facilidad para explicarlo — qué tan fácil es contarle a alguien qué hace, sin que sepa programar:

MARCA	SIGNIFICA
Alta	Se explica con una frase y se entiende a la primera
Media	Hace falta un ejemplo, pero con el ejemplo queda claro
Baja	Sólo se entiende explicando primero qué pasa si no existe

Las dos escalas son independientes, y ahí está lo interesante. **El grafo de relaciones fue difícil de construir y es facilísimo de explicar** (es un dibujo: se ve y se entiende). **La escritura atómica fue relativamente fácil de construir y es difícil de explicar**, porque su valor entero consiste en un desastre que nunca ocurre. Las piezas de dificultad alta y explicación baja son las que más necesitan estar documentadas: son las que alguien podría quitar por no verles el sentido.

5. El modelo de datos: 17 tablas

Es el corazón del sistema y el mejor lugar para empezar a leerlo. Todo lo demás —validación, pantallas, documentación, reportes— se genera a partir de aquí.

> **Dificultad: Alta · Facilidad para explicarlo: Media.** Difícil porque las decisiones de modelo son las únicas que no se pueden corregir barato después: cuando ya hay 194 filas apuntando a una llave, cambiarla es una migración. Media de explicar porque cada tabla por separado se entiende sola; lo que cuesta es explicar por qué están partidas así.

Las tablas y lo que contienen hoy

TABLA	FILAS	QUÉ ES
`personas`	243	Todos los seres humanos del sistema, sin importar su papel *(3 de prueba)*
`personas_roles`	172	Qué es cada quien: alumno, tutor, profesor, empleado *(3 de prueba)*
`alumnos`	115	Lo que sólo aplica a un alumno (programa, estado)
`relaciones`	194	Quién es qué de quién: tutor, contacto de emergencia, paga por
`alumnos_instrumentos`	129	Qué estudia cada alumno
`identificadores_externos`	312	El puente entre nuestra llave y la de Jotform o Kydemy
`disponibilidad`	157	Cuándo puede cada persona, como semana tipo *(11 de prueba)*
`clases`	4	Qué día y a qué hora es cada clase *(hoy todas de prueba)*
`clases_profesores`	5	Quién la da *(de prueba)*
`clases_alumnos`	12	Quién la toma *(de prueba)*
`programas`	13	Los PDA con su cuota mensual
`descuentos`	7	Qué descuentos existen
`alumnos_descuentos`	115	A quién se le aplica cuál *(pendiente de capturar)*
`espacios`	10	Los salones reales de la escuela
`instrumentos`	15	El catálogo de instrumentos
`conceptos`	0	Catálogo de conceptos de cobro *(vacío todavía)*
`respuestas_inscripcion`	115	Los envíos de Jotform aplanados (capa `interim`)

Las cuatro decisiones que condicionan todo lo demás

Una persona es una persona. Un profesor que además es tutor de su hija es UNA fila en `personas` con dos roles, no dos personas. Duplicar humanos es la causa raíz de los pagos que no cuadran y de los mensajes que llegan dos veces. Ésta es la decisión que convirtió 312 menciones en 240 personas reales.

`alumnos` extiende a `personas`, no la duplica. Un alumno usa el mismo `persona_id` como llave, en vez de tener un `alumno_id` propio. Dos llaves para la misma persona terminan desincronizadas, siempre.

`identificadores_externos` es el puente. Guarda, para cada persona, cómo la llama cada sistema de fuera: Jotform, Kydemy, WhatsApp. Es la pieza diseñada específicamente para el cruce de las tres fuentes que planteaba el documento anterior, **sin adivinar por nombre**. Hoy tiene 312 filas: el rastro de cada mención hasta el envío que la originó.

`relaciones` incluye el tipo `paga_por`. Resuelve el caso que rompe cualquier cruce ingenuo de pagos, el que el propio usuario identificó desde el principio: *"Sofi paga por mí"*. Quien paga no siempre es el alumno, y el sistema tiene dónde registrarlo.

Lo que se decidió NO guardar

Tan importante como lo que se guarda. **Las familias no se guardan: se deducen.** Dos alumnos son hermanos si apuntan al mismo tutor; hay 10 familias con hermanos, 21 alumnos en total. Guardar filas de "hermano" sería materializar una inferencia que se queda desactualizada en cuanto cambie un tutor. Deducirla siempre acierta.

Lo mismo con los horarios: los tres estados que definió la escuela son disponible, bloqueado y *"ni lo uno ni lo otro"*. Ese tercero **es la ausencia de fila**. Guardarlo sería tener dos formas de decir lo mismo, y tarde o temprano se contradicen.

6. La capa que cuida los archivos

Un CSV no sabe defenderse: acepta cualquier cosa que le escribas. Esta capa es lo que compensa eso. Son cinco módulos, y entre ellos está la mayor parte de la ingeniería que no se ve.

Lectura y escritura (`io.py`)

Regla innegociable del proyecto: ninguna otra parte del código lee ni escribe `data/` directamente. Todo pasa por aquí, porque aquí viven las garantías: un teléfono no se convierte en número, un ID no pierde el cero de adelante, y las filas salen siempre en el mismo orden.

> **Dificultad: Media · Facilidad para explicarlo: Baja.** La parte difícil de explicar es la **escritura atómica**: el archivo se escribe primero en un temporal y sólo al final se reemplaza de golpe. Si algo falla a medio camino —se corta la luz, se llena el disco—, el archivo anterior sigue intacto. Su valor entero consiste en un desastre que nunca se ve, y por eso es exactamente el tipo de código que alguien quitaría por "innecesario".

Validación (`validate.py`)

`lisart validate` revisa seis cosas contra el esquema: que estén todas las columnas, que cada valor se pueda interpretar, que no haya vacíos donde no se permite, que la llave primaria no se repita, que los valores estén dentro del catálogo permitido, y que **lo que apunta a otra tabla exista de verdad**.

> **Dificultad: Media · Facilidad para explicarlo: Alta.** Se explica en una frase: *revisa que los datos cumplan lo que prometen*. Y se demuestra en un minuto: se rompe un dato a propósito y el sistema dice qué está mal **y en qué línea del archivo**. Eso se hizo: se introdujeron los cinco tipos de error y los detectó todos.

Respaldos (`backup.py`)

Antes de cada escritura desde la pantalla se guarda una copia del estado anterior en `data/backups/`. Tres decisiones dentro:

- Se respalda **sólo si el dato nuevo pasó la validación**, para que un intento fallido no llene la carpeta de copias inútiles.
- Se conservan las **últimas 50** por tabla: guardar datos personales de menores para siempre no es gratis en responsabilidad.
- **Restaurar también deja copia** de lo que pisa. Deshacer una restauración tiene que ser posible.

> **Dificultad: Baja · Facilidad para explicarlo: Alta.** *Nada se sobrescribe sin dejar copia.* Se entiende sin más.

Identidad (`ids.py`)

Genera las llaves (`PER-0001` , `ALD-0042`) y normaliza nombres y teléfonos para poder comparar. Es la pieza que decide si dos menciones son la misma persona.

> **Dificultad: Alta · Facilidad para explicarlo: Media.** Alta porque la deduplicación por nombre es un problema sin solución perfecta: Jotform no da identificadores de persona, así que lo único que hay es el nombre escrito por gente distinta con mayúsculas, acentos y espacios distintos. El criterio actual acierta en los 115 casos reales, pero se sabe que es provisional: **cuando entre Kydemy, la llave buena será** `identificadores_externos` , no el nombre.

El esquema (`schema.py` + `tables.py`)

`tables.py` es la fuente única de verdad del modelo. Las 17 tablas se describen ahí, y de esa descripción salen automáticamente: la validación, la documentación técnica, el mapa de Obsidian y **hasta los controles de la pantalla**. Agregar una columna al modelo la hace aparecer sola en la interfaz.

> **Dificultad: Media · Facilidad para explicarlo: Media.** La idea se explica bien con la consecuencia: *el modelo se escribe en un solo lugar, y todo lo demás se entera solo*. Sin esto, cada cambio de modelo habría que replicarlo a mano en cuatro sitios, y el cuarto siempre se olvida.

7. De dónde salen los datos

Jotform: las inscripciones

`lisart ingest jotform` baja todos los envíos, guarda copia cruda con fecha en `data/raw/jotform/`, los aplana a las **113 columnas** de `respuestas_inscripcion` y **sólo agrega lo que no estaba**. Si no hay nada nuevo, no toca el archivo.

Se baja todo cada vez, no sólo lo nuevo, a propósito: es la única forma de detectar envíos **editados o borrados** en Jotform. Se reportan, pero no se pisan sin permiso.

> **Dificultad: Alta · Facilidad para explicarlo: Media.**

El riesgo estructural que hay que conocer. Jotform identifica cada campo por su **posición** en el formulario, no por su nombre. Si alguien reordena o inserta una pregunta, los datos empiezan a caer en la columna equivocada **sin que nada truene**: los teléfonos aparecerían en la columna de correos y el sistema los guardaría tan contento. Por eso hay una revisión de mapeo que corre en cada ingesta y avisa. **Hay que hacerle caso a esos avisos.**

Esa revisión ya sirvió. En la primera corrida real detectó tres cosas:

- Los campos 70 y 71 del formulario **son dos cosas distintas**, no una. Se confirmó con los datos: los 55 que llenan el 70 son todos mayores de edad firmando por sí mismos; los 59 que llenan el 71 son todos menores, firmados por su tutor. El script anterior perdía el campo 70 completo.
- Existen campos que nadie estaba leyendo (comprobante de domicilio, fecha de nacimiento del contacto de emergencia).
- El formulario **no pregunta el parentesco**. Por eso esa columna está vacía en las 194 relaciones: no es un hueco del sistema, es una pregunta que falta.

El Excel de la escuela: precios y descuentos

`lisart ingest catalogos` lee `data/catalogos_lisart.xlsx` y escribe las tablas `programas` (13 con su cuota) y `descuentos` (7). **La escuela sigue editando su Excel**, que es lo importante: no se le pidió cambiar su forma de trabajar. El archivo se trata igual que Jotform —copia cruda con fecha— porque es una fuente externa más.

> **Dificultad: Baja · Facilidad para explicarlo: Alta.** *Su Excel entra al sistema; ustedes lo siguen editando igual.*

8. Los procesos: de datos crudos a respuestas

Alta de inscripciones

`lisart procesar inscripciones` convierte los envíos de Jotform en personas, alumnos y relaciones. Es donde vive la deduplicación, y es el proceso con más criterio de todo el sistema.

> **Dificultad: Alta · Facilidad para explicarlo: Media.**

De sus decisiones, la más ilustrativa es la de las fechas de nacimiento en conflicto. Lo obvio sería *"gana el dato más reciente"*. Se descartó al descubrir que en el bloque del tutor la gente captura la fecha del hijo. La regla que quedó es: **gana la fecha que la persona declaró de sí misma como alumno; si no se puede decidir, se deja vacía.** Un hueco se nota y se corrige; una fecha equivocada se propaga en silencio a la edad, al programa y al precio.

Resultado sobre datos reales: 312 menciones → 240 personas, 59 menores todos con tutor registrado, 10 familias con hermanos detectadas.

Roles

`procesos/roles.py` contesta dos preguntas que van en direcciones opuestas: *¿quiénes son los tutores?* (un directorio) y *¿qué es esta persona?* (un campo de su ficha).

> **Dificultad: Baja · Facilidad para explicarlo: Alta.**

Con una decisión que vale la pena: **quitar un rol no lo borra, lo cierra** con fecha de fin. Borrarlo haría imposible entender por qué una clase de marzo estaba asignada a alguien que "nunca fue profesor". Volver a darlo reabre la misma fila, no crea otra.

Horarios y reposición

Cuatro tablas y un módulo que responde: cuándo puede alguien, y qué huecos comparten un profesor y un alumno para reponer una clase.

> **Dificultad: Alta · Facilidad para explicarlo: Media.**

Dos decisiones que se sostienen en un criterio de costo, no de elegancia:

- **Bloqueado le gana a disponible** cuando dos rangos se traslapan. El costo de equivocarse no es simétrico: ofrecer una hora ocupada manda a un alumno a una clase que no existe; no ofrecer una hora libre sólo desperdicia un hueco.
- **Se guardan rangos, no casillas.** "Lunes de 9 a 14" es una fila, no diez.

Lo que falta aquí es dato, no código. No hay ni un profesor ni una clase real en la base, así que la reposición hoy sólo devuelve resultados con los datos de prueba. El circuito completo ya funciona; le faltan los datos reales.

Descuentos

El catálogo decía qué descuentos existen; faltaba **a quién se le aplica cuál**. Ese dato no está en Jotform, no está en el Excel y no se deduce de nada: lo decide la administración caso por caso.

> **Dificultad: Media · Facilidad para explicarlo: Alta.**

`alumnos_descuentos` es **la única tabla del sistema pensada para abrirse en Excel**. Lista a los 115 alumnos, uno por renglón; se escribe el nombre del descuento en su celda, se guarda, y `lisart descuentos` completa lo demás. Se puede incluso escribir sólo el **nombre** del alumno: el comando lo resuelve, y si hay dos que se llaman igual **pregunta en vez de elegir**.

Tres reglas dentro, todas discutidas y decididas por el usuario:

- **Manda la celda, no el catálogo.** Se puede pactar un caso particular sin inventar una entrada de catálogo. El costo, asumido: cambiar el descuento familiar del 5 % al 7 % obliga a tocar los renglones que lo tengan. El comando avisa cuando una fila se separó del catálogo.
- **El porcentaje va en puntos** (5 = 5 %, 100 = beca completa). Resuelve un problema real del catálogo, que mezclaba `0.05` y `100.00` en la misma columna. La regla que las separa —*nadie da un descuento de 1 % o menos*— queda escrita en un archivo que una persona revisa.
- **Un descuento mal escrito se reporta y no se escribe nada**, para poder corregirlo en el mismo archivo.

Proyección de pago

`lisart proyeccion` cruza alumnos, programas y descuentos y escribe a `exports/` tres CSV: el detalle alumno por alumno, el resumen por programa y la lista de quienes se quedaron sin precio. Con `--excel` escribe además el archivo en el **formato mensual que ya usa la escuela**, con sus columnas de descuento ya llenas.

> **Dificultad: Media · Facilidad para explicarlo: Alta.** Es la pieza que mejor se explica de todo el sistema, porque su salida es una cifra que a la escuela le importa.

Resultado de hoy: $148,655.00 mensuales sobre 98 de 115 alumnos (85 %).

El 15 % restante no se estima ni se inventa: son 17 alumnos del programa "Individual (1xsemana)", que **no tiene precio en el catálogo**. Salen listados aparte y el total declara lo que le falta. Poner ese precio en el Excel es lo único que separa la proyección de cubrir al 100 %.

El amarre entre el texto del formulario y el catálogo está **escrito a mano y a la vista** en el código, en vez de adivinarse por parecido de texto. Es deliberado: una proyección de dinero no se construye sobre coincidencias aproximadas.

Datos de prueba

`lisart demo` siembra profesores, clases e inscripciones para poder ejercitar la agenda sin capturar nada a mano. **Todo lo sembrado lleva** `fuente = "demo"`, se ve en la pantalla y se borra entero con `lisart demo --limpiar` sin tocar un dato real.

> **Dificultad: Baja · Facilidad para explicarlo: Alta.**

9. Las pantallas

Son dos aplicaciones distintas, a petición de la escuela, y comparten todo lo de abajo: el mismo modelo, la misma validación, la misma escritura con respaldo.

La pantalla principal (`lisart ui`)

Una sección por tabla, un registro a la vez, con flechas para recorrerlos y buscador. Es la forma de que alguien que no vive en una terminal pueda consultar y corregir datos.

> **Dificultad: Alta · Facilidad para explicarlo: Alta.** Difícil por el entorno (ver la sección 12); fácil de explicar porque se ve.

Tres decisiones que la sostienen:

- **Los controles se generan desde el modelo.** El tipo del dato decide el control, un catálogo se vuelve lista desplegable y una llave foránea se vuelve un selector de valores existentes: **es imposible escribir una referencia rota**. Agregar una columna al modelo la hace aparecer sola.
- **No se puede editar lo que no se debe:** llaves primarias, marcas de tiempo y todas las tablas de capa `interim`, que se regeneran desde `raw` y editarlas sería trabajo perdido. La pantalla lo dice en pantalla, no lo esconde.
- **Los registros se titulan en lenguaje de personas.** Una relación no se muestra como `PER-0042 → PER-0117` sino como "Ana Torres — tutor → Luis Torres".

La pantalla de profesores (`lisart ui --profesores`)

Se entra **por la clase**, no por la persona: clase → profesor que la da → su semana. Quien administra la docencia no piensa en personas sueltas, piensa en "el piano de los jueves".

Incluye una aproximación que pidió la escuela: las horas que los alumnos de una clase marcaron se pintan como ocupadas para el profesor, **con trama rayada** para no confundirlas con un bloqueo capturado a mano. La escuela sabe que no es exacta —un alumno marca cuándo *puede* venir, no cuándo tiene clase— y aun así sirve mientras no exista el dato real. **No se guarda: es un cálculo**, y guardarlo quedaría desfasado en cuanto un alumno cambiara su horario.

El grafo de relaciones

Contesta *"¿qué alumnos se parecen?"*: los que estudian el mismo instrumento se agrupan a su alrededor, los hermanos alrededor de su tutor. 177 nodos.

> **Dificultad: Alta · Facilidad para explicarlo: Alta.** El mejor ejemplo de las dos escalas separadas: costó muchísimo y se explica señalando la pantalla.

Cada nodo es una figura musical, y la figura va además del color: alumno es negra, tutor es corchea, profesor es blanca, instrumento es redonda. Que la forma acompañe al color no es estética: es lo que hace que el dibujo siga funcionando para quien no distingue dos colores, y lo que permite leerlo impreso en blanco y negro.

El acomodo es explícito, no automático. El algoritmo estándar empujaba los instrumentos a los bordes y apelmazaba el centro: disperso y apretado a la vez. Ahora los instrumentos se colocan primero y los alumnos van en anillo alrededor del suyo; quien estudia dos instrumentos **se coloca entre los dos**, y en el dibujo se ve como el puente que es.

El diseño

La primera versión de la pantalla se veía genérica, y el usuario lo dijo sin rodeos. La segunda tiene una dirección explícita: **partitura impresa**. Tinta negra sobre papel; el vino de la paleta de Lisart es *el lápiz rojo con el que un músico anota su partitura* y por eso marca el registro actual y el botón de guardar; el oro **no se reparte**, vive en la banda de marca y nada más.

La firma de la interfaz es que **el navegador es un pentagrama**: la nota roja es el registro abierto y las grises sus vecinos. Toda la app consiste en recorrer registros de uno en uno, que es lo que hace una melodía. La altura de cada nota sale de una melodía fija indexada por número de registro —determinista, no aleatoria: si se sorteara, la misma persona cambiaría de nota en cada refresco y se vería como un parpadeo.

> **Dificultad: Media · Facilidad para explicarlo: Media.** El diseño es fácil de mostrar y difícil de justificar en abstracto. La prueba es empírica: se revisó **mirándolo**, con capturas de pantalla reales en cada iteración, y así salieron defectos que ninguna prueba automática detecta.

10. La documentación, que se genera sola

Seis documentos: dos se generan enteros desde el código, y de los otros cuatro se escribe el texto a mano y **el PDF se regenera con un comando**. Es deliberado: la documentación que se mantiene a mano se desfasa, y la desfasada es peor que ninguna porque se le cree.

DOCUMENTO	CÓMO SE MANTIENE
`docs/modelo_de_datos.md`	**Generado** desde el modelo. `lisart docs`
`vault/` (mapa de Obsidian)	**Generado.** `lisart vault`
`docs/guia_de_uso.md` + PDF	A mano el texto; el PDF **generado**. `lisart guia`
`docs/documentacion_general.md` + PDF	Este documento. Igual que la guía
`docs/mapa_del_codigo.md` + PDF	El documento para quien recibe el código. Igual que la guía
`docs/guia_de_entrega.md` + PDF	Cómo se le pasa el sistema a otro desarrollador. Igual que la guía

El vault de Obsidian

`lisart vault` genera un mapa navegable del sistema: cada tabla, cada fuente y cada proyecto es una nota, y las notas se enlazan entre sí. Se abre en Obsidian y se ve el sistema **como grafo**.

> **Dificultad: Media · Facilidad para explicarlo: Alta.**

Tiene un truco de diseño que vale la pena señalar: los proyectos enlazan a tablas **que todavía no existen** (`pagos`, `envios`). En el grafo salen como círculos huecos, así que **el mapa mismo es la lista de lo que falta construir**.

Y una regla que se respeta estrictamente: `vault/Tablas/` se borra y se reescribe completo en cada corrida, para que una tabla eliminada no deje nota huérfana; `Fuentes/`, `Documentos/` y `Proyectos/` se siembran una vez y después son del usuario, y el comando **no los pisa**.

La guía de uso y este documento

Los dos siguen el mismo mecanismo: el markdown es el original, el PDF es la copia que se entrega. El PDF nunca se edita, se regenera —dos documentos que se editan por separado se contradicen en la segunda semana, y el equivocado va a ser justo el que alguien tenga impreso.

> **Dificultad: Media · Facilidad para explicarlo: Alta.**

El detalle que hace que la regla se cumpla sola: al generar el PDF se guarda la **huella** del markdown que lo produjo. `lisart guia --check` la recalcula, y corre dentro de la revisión estándar del proyecto. Editar el

texto y olvidar el PDF **truena la revisión** en vez de pasar inadvertido. Sin eso, la regla dependía de que alguien se acordara.

El PDF se arma con el mismo diseño de la pantalla, y con las fuentes y el logo **incrustados en el archivo**: sale idéntico sin internet, hoy y en un año. Hay una prueba automática que verifica que no contiene ni una sola dirección remota.

11. Cómo se protege el dato

Los datos incluyen nombres, domicilios, teléfonos, firmas e identificaciones de **menores de edad**. Eso condiciona varias decisiones del sistema.

MEDIDA	QUÉ PROTEGE
`data/**` fuera del control de versiones	Los datos personales nunca salen en un repositorio
Las llaves de API en `.env`	No quedan escritas en el código ni en el historial
Sólo los catálogos se versionan	No llevan datos personales
Escritura atómica	Un corte de luz no deja un archivo a medias
Respaldo antes de escribir	Un error de captura se puede deshacer
Máximo 50 copias por tabla	No se acumulan datos de menores para siempre
Aviso antes de compartir `exports/`	La regla escrita: revisar qué columnas lleva

Hay un riesgo abierto y está declarado como la prioridad número uno del proyecto: la llave de API de Jotform es la misma que estuvo escrita directamente dentro del notebook original, y con ella ya se bajaron los datos de 115 menores. Rotarla es lo primero que hay que hacer, y no depende de nadie más. El notebook todavía la contiene, así que además hay que hacerlo **antes** de poner el proyecto bajo control de versiones: si se guarda con la llave dentro, queda en el historial para siempre.

> **Dificultad: Baja · Facilidad para explicarlo: Alta.** Rotar una llave es cambiar una contraseña. Lo difícil no es hacerlo, es acordarse de que está pendiente. Por eso está escrito aquí.

12. Dónde estuvo lo difícil de verdad

Esta sección existe para que el conocimiento comprado con horas no se pierda. Es lo que un desarrollador nuevo necesitaría saber para no volver a tropezar, y está también dentro del repositorio para que lo lea quien lo mantenga.

Ninguna librería de gráficas funciona en este equipo. Las tres opciones estándar de Streamlit —plotly, altair y hasta sus gráficos nativos— salen **vacías, sin ningún error**. Se comprobó con una aplicación mínima. Por eso el grafo y el pentagrama están dibujados a mano, elemento por elemento. No es capricho: es la única vía que funciona aquí.

Y de las tres formas de mostrar un dibujo, sólo una sirve. Una lo borra al "limpiarlo" por seguridad; otra lo pinta rápido pero lo encierra en una caja que **bloquea el clic** en los nodos; la tercera funciona, pero es lenta. Se eligió la lenta a conciencia: la sección del grafo tarda cerca de un minuto en pintar, y el clic vale más que el minuto.

Un elemento que "no se pinta" puede simplemente no haber terminado. Una sección puede tardar ~30 segundos en aparecer completa. Capturar la pantalla a los 6 segundos y ver que falta un bloque hace creer que se perdió. Se llegó a reescribir un panel entero por ese diagnóstico equivocado. **La regla que quedó: antes de concluir que algo falta, revisar si el proceso sigue corriendo.**

Un dato faltante en pandas no es ni verdadero ni falso. Preguntar `si la celda tiene valor` sobre una celda vacía **revienta** con un error críptico, en vez de responder "no". Estaba en las dos pantallas y en una función que usan varios procesos. Se corrigió con una función propia y hay pruebas de regresión. Es la trampa número uno del proyecto y está escrita en las reglas del repositorio.

Un bug que sólo se ve navegando. Los controles del formulario se nombraban igual en todos los registros, y la pantalla guarda el estado de un control por su nombre: al avanzar, la tarjeta mostraba a una persona pero los campos seguían con los datos de la anterior. **Guardar habría sobrescrito a una con la otra.** No lo detectó ninguna prueba: se vio tomando capturas mientras se navegaba. Hoy hay una prueba que lo cubre.

Un comando que se mata a sí mismo. El comando habitual para cerrar la aplicación se cierra a sí mismo antes de llegar a ella, porque su propio nombre coincide con lo que busca. Parece trivial y costó una tarde.

Un error que aparecía sólo fuera del repositorio. Si los datos viven en otra carpeta —que es justo lo que hace quien no quiere datos de menores en una carpeta sincronizada—, la proyección tronaba **después** de haber escrito los archivos: el trabajo hecho y aun así un error en pantalla. Estaba también en la pantalla al guardar. Lo encontró correr la guía entera contra una copia real de los datos.

Y la lección de método: se ejecutó comando por comando todo lo que la guía dice que se puede hacer, contra una copia completa de los datos, y se levantaron las dos pantallas de verdad. **Salieron tres defectos reales que ninguna prueba automática había detectado.** Las pruebas verifican que el código hace lo que dice; sólo usarlo verifica que sirve.

13. Qué está verificado y qué no

Lo importante de esta sección es la segunda columna. Un sistema que declara sus límites es un sistema en el que se puede confiar dentro de ellos.

Verificado, corriendo de verdad:

- **208 pruebas automáticas** pasan: 183 rápidas y 25 de pantalla, que tardan unos cinco minutos y por eso se corren aparte. Cubren el modelo, la validación, la ingesta, los procesos y la interfaz.
- `lisart validate` **responde "todo en orden"** sobre las 17 tablas con datos reales.
- Se **rompieron datos a propósito** y el validador detectó los cinco tipos de error, con el número de línea del archivo.
- Se corrió la **API real de Jotform** y de ahí salieron los 115 alumnos y las 240 personas reales que hay hoy (las otras 3 son de prueba).
- Se corrió **la guía completa, comando por comando**, contra una copia de los datos, y se levantaron las dos pantallas con capturas.
- La proyección se verificó a mano: con descuentos simulados bajó de $148,655.00 a $146,985.25 con el desglose correcto.
- El PDF se revisó **mirándolo** página por página, no sólo generándolo. Así salieron dos defectos de maquetación.

No verificado, y conviene saberlo:

- **Los diagramas del vault no se han visto renderizados** en Obsidian. El texto se generó y se validó; nadie ha abierto la aplicación para confirmarlo.
- **La ingesta automática semanal no está agendada.** El comando existe y funciona; no hay ninguna tarea programada que lo corra sola.
- **La reposición no se ha probado con datos reales**, porque no hay profesores ni clases reales. Funciona con los de prueba.
- **Los descuentos están sin capturar.** El archivo existe con los 115 alumnos y las celdas vacías; llenarlo es trabajo de la administración.

14. Estado actual y siguientes pasos

Dónde está parado el sistema

La base de alumnos está terminada y es real. 240 personas, 115 alumnos, 194 relaciones, con su instrumento, su programa y su horario declarado. Sobre esa base ya se calcula una proyección de pago.

Lo que falta es de tres tipos, y conviene no mezclarlos porque se destraban de formas distintas:

1. **Datos que sólo la escuela puede dar.** Baratos, y frenan cosas.
2. **Una fuente que no está conectada** (Kydemy). Desbloquea tres pendientes de golpe.
3. **Trabajo de desarrollo**, que ya está diseñado pero no escrito.

Los pendientes, por prioridad

PRIORIDAD	PENDIENTE	POR QUÉ AHÍ
1	Rotar la llave de API de Jotform	Riesgo abierto, no depende de nadie más
2	Poner el proyecto bajo control de versiones	Hoy un borrado accidental se lleva todo. Después de rotar la llave
3	Precio de "Individual (1xsemana)" en el Excel	Es lo único que separa la proyección de cubrir al 100 %
4	Capturar los descuentos por alumno	El archivo está listo; es captura, no código
5	Conector de Kydemy	Trae los profesores, verifica los pagos y genera comprobantes
6	Tabla de pagos reales	Convierte la proyección en conciliación
7	Agendar la ingesta semanal	El comando existe; falta programarlo
8	Nombres reales y capacidad de los salones	Van a salir en mensajes y horarios

El siguiente objetivo declarado

El que planteó el usuario: **automatizar la proyección de pago y la generación de comprobantes**. La primera mitad ya está: la proyección se calcula sola. La segunda mitad depende de Kydemy.

El método que hace falta cerrar es el cruce de tres fuentes —la conversación de "Pagos Lisart", Kydemy y la proyección— para saber quién ya pagó. El sistema ya tiene las dos piezas que ese cruce necesita y que no se podían improvisar: `identificadores_externos`, para no cruzar por nombre, y el tipo de relación `paga_por`, para el caso "Sofi paga por mí".

Cómo encaja con el resto de los frentes de Lisart

El documento anterior identificaba cinco frentes. Así queda el mapa con esto ya construido:

FRENTE	CÓMO LO TOCA ESTE SISTEMA
Formularios (Jotform)	Sigue siendo la puerta de entrada. Ahora tiene a dónde llegar
Centralización de pagos	La proyección es la mitad de arriba. Falta el registro de pagos reales
Mensajes por WhatsApp	Aquí está a quién mandarle y cuánto debe: los datos que le faltaban
Segmentación de peticiones	Sin tocar. Entraría como una fuente más, igual que Jotform
CRM / sistema integral	Si algún día se adopta, este modelo es lo que se le carga

Y contesta la recomendación número dos de aquel documento —*definir una estructura central de datos*— que era la que bloqueaba a las demás.

15. Palabras que se usan aquí

PALABRA	QUÉ QUIERE DECIR
Terminal	La ventana donde se escriben comandos en vez de dar clic
Comando	Una instrucción que se escribe en la terminal. Aquí todos empiezan con `lisart`
CSV	Un archivo de tabla, en texto plano. Se abre en Excel
Capa	Una etapa del camino del dato: `raw`, `interim`, `master`, `exports`
Tabla	Un archivo CSV con reglas: qué columnas tiene y qué se permite en cada una
Llave primaria	La columna que identifica cada renglón sin repetirse (`PER-0042`)
Llave foránea	Una columna que apunta a un renglón de otra tabla
Catálogo	Una tabla de valores permitidos, mantenida por una persona
Validar	Revisar que los datos cumplan las reglas. `lisart validate`
Ingesta	Traer datos de una fuente de fuera al sistema
Escritura atómica	Guardar de forma que el archivo quede completo o no se toque
Deduplicar	Darse cuenta de que dos renglones son la misma persona
Proyección	Cuánto *debería* entrar este mes. No es lo que entró
Vault	La carpeta que se abre en Obsidian y muestra el sistema como mapa
Prueba automática	Código que revisa que el código funcione. Corre solo

16. Conclusión

Lo que se construyó no es un conjunto de scripts: es **la estructura central de datos** que el diagnóstico anterior señalaba como la pieza faltante, y sobre la que se apoyan los demás frentes.

Su valor no está en la cantidad de código —son diez mil líneas, que no es mucho— sino en dos cosas que no se ven en una demostración. La primera es que **el sistema se niega a hacer algo dudoso**: si un dato no cuadra, se detiene y lo dice. La segunda es que **cada decisión está escrita con su porqué**, dentro del propio repositorio, de modo que quien lo herede pueda cambiarla sabiendo qué se rompe.

El estado es honesto: la base de alumnos está terminada y es real; la de pagos y profesores no existe todavía, y se sabe exactamente qué falta para que exista. Lo que ya no hace falta es decidir cómo guardar las cosas. Esa parte está resuelta, probada y documentada, y las tablas que faltan heredan gratis la validación, la documentación y los respaldos con sólo describirlas.

El siguiente tramo no es de arquitectura, es de conexión: traer Kydemy, capturar lo que sólo la escuela sabe, y cerrar el circuito de pagos. Todo lo que ese tramo necesita como cimiento ya está puesto.